

# 数据库系统复习

## 1.引言

### 1.什么是数据库

数据由称之为“数据库管理系统(DBMS)”统一管理和控制。

数据库管理系统(DBMS): 专门管理和维护大量数据的系统软件。例: PostgreSQL、GaussDB、MySQL、Oracle、SQL Server、DB2

数据库系统(DBS): 有管理数据库功能的计算机系统。

数据库是长期存储在计算机内、有组织的、统一管理的、可共享的相关数据的集合。

### 2.为什么使用数据库

优点: 简单、方便、共享、高效、安全

模式: 表模式为表名(列名1, 列名2...)

实例: 表内的具体全部数据

数据库管理系统的一个主要作用就是隐藏关于数据存储和维护的某些细节, 即不同的使用者仅能观察到自己必要的数据库。

数据库模式可以分为:

物理模式(内模式)

逻辑模式(简称模式)

视图模式(子模式、外模式)

概念模式

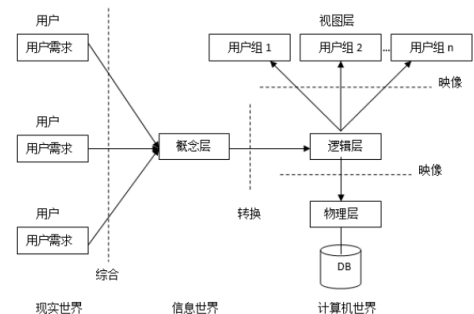


图 1-1 数据抽象的层次

### 3.三层模式两级映射

三层模式:

(1) 第一层模式, 即外模式。

外模式又称为子模式或用户模式, 它是数据库用户与数据库系统的接口, 是数据库用户看到的数据视图。

一个数据库可以有多个外模式, 应用程序都是和外模式打交道的, 外模式是保证数据库安全性的一个有力措施。

每个用户只能看见和访问所对应的外模式中的数据, 数据库中的其余数据对他们是不可见的。也就是说, 数据库用户只能通过外模式这个窗口, 看到数据库是什么。

(2) 第二层模式, 即模式。

模式是所有数据库用户的公共数据视图(全局视图), 是数据库中全体数据的逻辑结构和特征的描述。

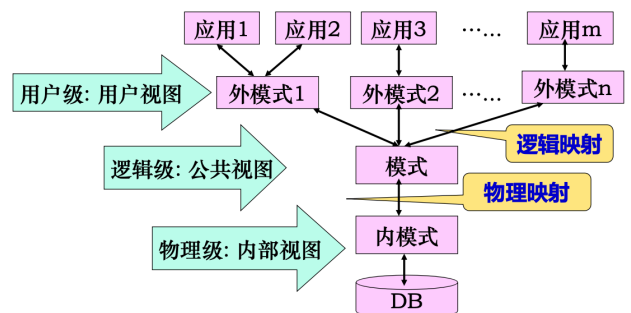
一个数据库只有一个模式, 定义模式时不仅要定义数据的逻辑结构, 而且要定义数据之间的联系, 定义与数据有关的安全性、完整性要求。

所有数据库用户看到各自的数据视图, 都是这个公共数据视图的一部分。

(3) 第三层模式, 即内模式。

一个数据库只有一个内模式, 内模式又称为存储模式, 它是数据库物理结构和存储方式的描述, 是数据在数据库内部的表示方式。

数据库管理系统支持一个物理模式、一个逻辑模式和多个视图模式。



两级映射：

(1) 第一级映射，即外模式与模式之间的映射。

对应于同一个模式可以有任意多个外模式。对于每一个外模式，数据库系统都有一个外模式 / 模式映射，它定义了该外模式与模式之间的对应关系。

在模式改变时，由数据库管理员对各个外模式-模式映射作相应的改变，可以使外模式保持不变。由于应用程序是依据数据的外模式编写的，所以应用程序可以不必修改，从而保证了数据与程序之间的逻辑独立性。

(2) 第二级映射，即模式与内模式之间的映射。

数据库中只有一个模式，也只有一个内模式，所以模式 / 内模式映射是唯一的，它定义了数据库的全局逻辑结构与全局存储结构之间的对应关系。

当数据库的存储结构改变时，由数据库管理员DBA对模式/内模式的映射也作相应改变，所以可以使模式保持不变，应用程序也不必修改，从而保证了数据与程序的物理独立性。

4.数据独立性：

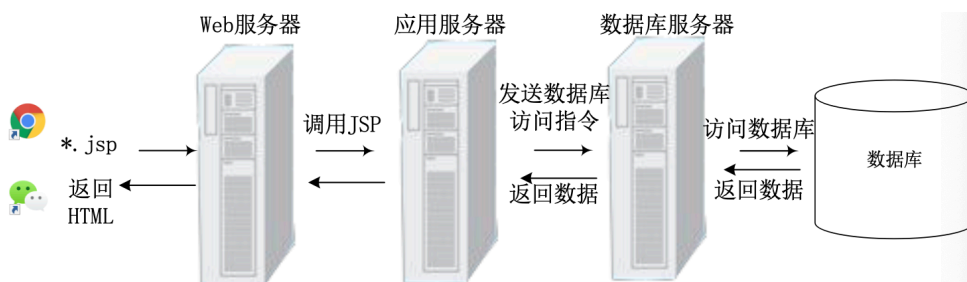
在某个层次上修改数据库的模式定义而不影响位于其上层模式的能力叫做数据独立性。

物理数据独立性：是指修改数据库的物理模式而不必重写应用程序的能力；

逻辑数据独立性：是指修改数据库的逻辑模式而不必重写应用程序的能力；

数据库系统的三层模式与两级映射保证了数据的独立性，适应变化。

5.数据库系统是如何工作的？



## 2.关系模型

1.关系数据库的结构

关系数据库是表的集合，每个表有唯一的名字。列首被称为属性。每个属性有一个允许值的集合，称为该属性的域或者取值范围、数据类型。关系中元组次序无关紧要。

关系的键：

(1) 超键：在给定关系R中，能惟一标识元组的属性集合

(2) 候选键：给定关系R中，能够唯一标识出各个元组的属性集合，并且不含多余属性

(3) 主键：用于标识元组的候选键。主键具有唯一性。主键需要非空且唯一

(4) 外键：关系t1的一个属性子集S，是另一个关系t2的主键，则称S是关系t1的外键。外键的值，要么和被参照关系主键的值对应，要么为空值。

2.关系数据库的操作

(1) 关系代数



关系代数包括一个运算集合。这些运算以一个或两个关系作为输入，产生一个新的关系作为结果。

选择运算：给出满足给定谓词条件的元组。

表示：将谓词写作的下标，并在后的括号中给出作为参数的关系。

投影运算：所有希望在结果关系中出现的属性

表示：所有希望在结果关系中出现的属性作为下标，而作为参数的关系跟随在后的括号中。

结果：返回作为参数的那个关系的部分属性构成的新关系；重复的行均被去除

并运算：将属性相同的两个表求并不存在的元组进行加入

集合差运算：用“-”表示的集合差运算找出在一个关系中而不在另一个关系中的那些元组。

要求：和并运算一样，集合差运算也必须在相容的关系间进行。

笛卡尔积运算：用“x”表示的笛卡尔积运算可以将任意两个关系的信息组合在一起。

结果：运算符左侧关系中的每一个元组与右侧关系的每一个元组拼接，形成结果关系中的一个元组

要求：属性名称前加上该属性来自的关系名称，用小数点分割（可以简化）

例：S = (examiner.erid, examiner.ername, examiner.ersex, examiner.erage, examiner.dname, erexam.erid, erexam.eid)

命名计算：用小写希腊字母 $\rho$ 表示的命名运算可对关系更名，或赋予关系代数运算结果一个名字

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

**返回表达式E的结果，并赋给它名字x，同时将E的各属性更名为 $A_1, A_2, \dots, A_n$ 。**

集合交运算：集合交运算用符号 $\cap$ 来表示，表达式 $t_1 \cap t_2$ 的结果是由那些既在关系 $t_1$ 中又在关系 $t_2$ 中的元组组成；

要求：集合交运算也必须在相容的关系间进行。

自然连接运算：可以将某些选择运算和笛卡尔积运算合并为一个运算，用“连接”符号 $\Join$ 来表示：即 $t_1 \Join t_2$

计算过程：首先形成它的两个参数的笛卡尔积；然后在笛卡尔积的结果上，基于两个参数的关系模式中都出现的属性，进行属性值相等的选择运算；最后还要去除重复属性（列）。

性质：满足结合律

除运算：除运算用符号“ $\div$ ”表示，适合于包含诸如“所有的”此类短语的查询。

要点1：模式相减

要点2： $t_2 \times t_3$  属于  $t_1$  乘不溢出的最大 $t_3$

赋值运算：赋值运算用符号“ $\leftarrow$ ”来表示

外连接：自然连接会损失一些信息

左外连接，用符号 $\Join_L$ 来表示；右外连接，用符号 $\Join_R$ 来表示；全外连接，用符号 $\Join_F$ 来表示。

过程：先自然连接，再在左侧(右侧、两侧)关系中找出那些没有在连接结果中出现的元组；把这些元组也加入到连接结果中去，并且用空值null来填充那些来自右侧(左侧、相对一侧)关系的属性的值；

聚集函数：诸如sum、avg、count、max和min此类的函数。它们的输入是一系列值的集合，而返回的结果则是单个值的集合。

**G<sub>avg(erage)</sub>(examiner)**

分组聚集：对关系中的元组按某一条件进行分组，并在分组的元组上使用聚集函数。

## 6.数据库设计：实体联系方法

### 1.E-R图

矩形表示实体、椭圆表示属性、主键加下划线

能够并且用以区分一个实体集中不同实体的最小属性集称为实体标识符或称为实体主键，组成主键的属性称为标识属性

联系表示一个或多个实体之间的关联关系。联系用菱形框表示。可以有一元联系、二元联系、三元联系

联系的约束：

对于二元联系，可能的映射基数有1:1，1:N，M:N等三种。

一对一

如果实体集E1中每个实体至多和实体集E2中的一个实体有联系，反之亦然，那么实体集E1和E2的联系称为“一对一联系”，记为“1:1”。

一对多

如果实体集E1中每个实体可以与实体集E2中任意个（零个或多个）实体间有联系，而E2中每个实体至多和E1中一个实体有联系，那么称E1对E2的联系是“一对多”联系，记为“1:N”。

多对多

如果实体集E1中每个实体可以与实体集E2中任意个（零个或多个）实体有联系，反之亦然，那么称E1和E2的联系是“多对多联系”，记为“M:N”。

最小映射基数min最大映射基数max

完全参与：用双线边 部分参与：用单线边

### 2.E-R模型转换为关系模型

一般原则：

① 一个实体转换为一个关系模式；实体的一个属性变换为该表的一个列；实体的主键就是表的主键。

② 一个联系可转换为一个关系模式；联系的属性对应表的属性，另外并上所有参与联系的各实体的主键；

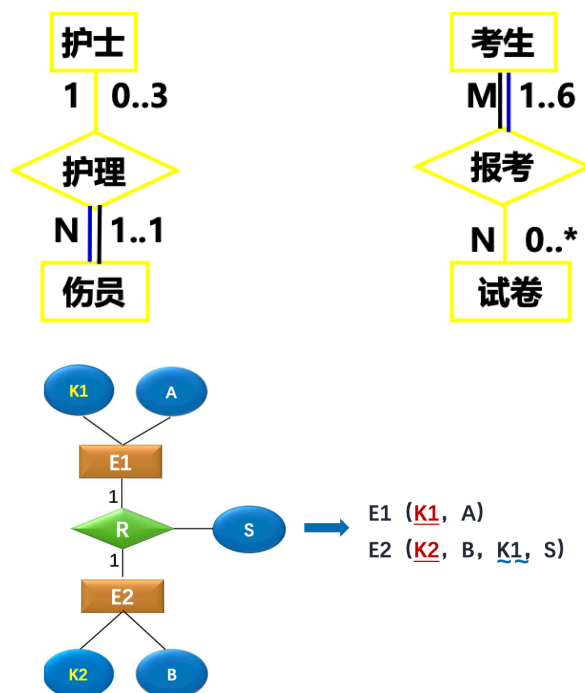
M: N联系表的主键是所有参与联系的各实体主键的并集。

1: N联系表的主键是多端实体主键。

1: 1联系表的主键是任一端实体主键。

③ 主键相同的表可合并

一个1: 1联系转换的表可与任一端实体对应的表合并



一个1: n联系转换的表可与n端对应的表合并

### 3.基本E-R图的设计

(1) 属性：简单属性、复合属性、单值属性、多值属性

简单属性和复合属性：复合属性（属性还有分解）

单值属性和多值属性：多值属性（可能有多个值，如联系方式），使用双线椭圆表示，实现时拆分存储属性和派生属性：派生属性不必存储可以推导出，使用虚线椭圆和虚线表示

空值属性：空值可以表示值无意义、缺失（知存在）、不确定是否存在

(2) 联系：一个联系涉及到的实体集的个数，称为该联系的元数或度数

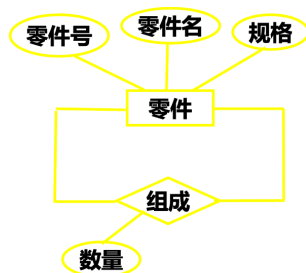
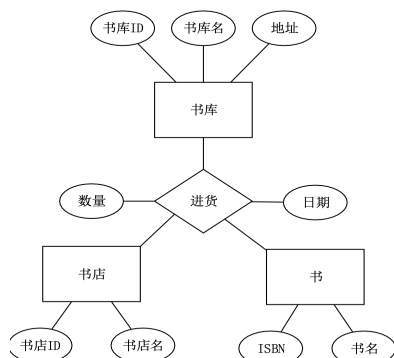
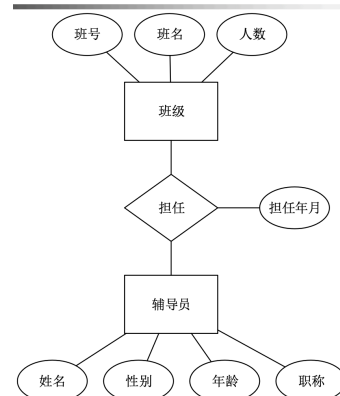


图 一元联系



图的设计——二元联系

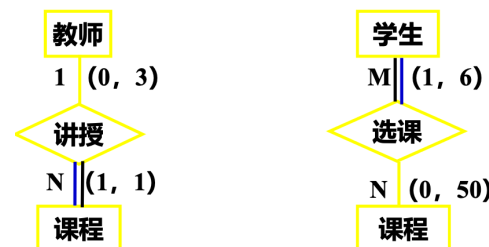


联系约束：限制参与联系的实体的数目

基数约束：实体集E1和E2之间有二元联系，则参与一个联系中的实体数目称为映射基数。

对于二元联系类型，可能的映射基数有1:1, 1:N, M:N, M:1等四种。

有时我们对映射基数还要作出更精确的描述，即对参与联系的实体数目指明相关的最小映射基数min和最大映射基数max，用范围“min..max”的方式表示。若上界没有限制，即“\*”表示



参与约束：如果实体集E中的每个实体都参与联系集R的至少一个联系，

我们称实体集E“完全参与”联系集R。如果实体集E中只有部分实体参与联系集R的联系中，我们称实体集E“部分参与”联系集R。在ER图中表示时，完全参与用双线边表示，部分参与用单线边表示。

(3) 强实体与弱实体

强实体：其属性可形成主键的实体

弱实体：其所有属性都不足以形成主键的实体

弱实体使用双线矩形框表示，与弱实体关联的联系，采用双线菱形，边使用双线。

标识属性使用单虚线标注。

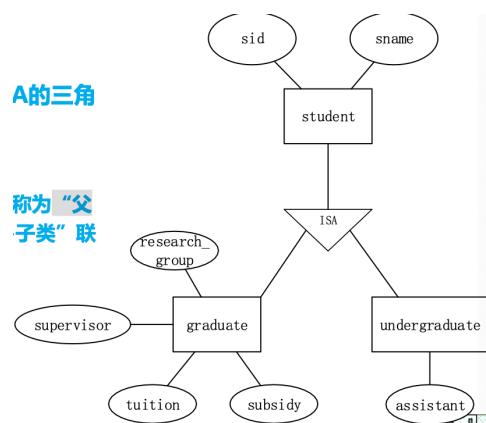
在含弱实体E-R图向关系模式转换的一般规则：

弱实体联系和弱实体一起转换成一个关系模式，并且需要加入强实体的主键。

如果弱实体在联系中的基数约束处于多端，则主键是强实体主键+弱实体标识属性。

### 4.特殊化

在E-R图中，特殊化通过标记为ISA的三角形构件“▽”来表示，“父



类-子类”联系集。

特殊化是自顶向下的设计过程：对初始实体集求精，产生一系列不同层次的实体子集；

在这个过程中，实体间的区别被明显地表达出来。

一般化是自底向上的：根据实体间共同具有的特征，将多个实体集综合成一个较高层次的实体集；

实体集间存在的共性通过一般化来表达。

同高层实体集相联系的所有属性和联系集也适用于它的所有低层实体集；

低层实体集所特有的属性和联系集仅仅适用于某个特定的低层实体集。

当父子实体转化为关系模式时：

第一种方式：为高层实体集创建一个表，为每个低层创建一个表并加入高层实体集逐渐属性，并且设为外键。

第二种方式：不给出高层实体集对应的表，只为低层创建。

## 5.E-R图的转换

转换的一般原则：

1)每个实体型转换为一个关系模式

2)两个(多个)实体间的n:m联系转换为一个关系模式

3) 一个1:1联系应与某一端的关系模式合并。一个1:n联系应与n端的关系模式合并。

4)相同码的关系模式应合并。

## 7.属性-联系方法

### 1.数据依赖

全键：关系模式的候选键由整个属性组构成

主属性：候选键中的属性，称作主属性

非主属性：不包含在任何候选键中的属性称为非主属性或非键属性

逻辑蕴涵：如果给定关系模式S的函数依赖集D，可以证明其他某些函数依赖也成立，就称这些函数依赖被D逻辑蕴涵；

闭包：如果给定关系模式S的函数依赖集D，D的闭包是指被D所逻辑蕴涵的所有函数依赖的集合，记为 $D^+$ 。

Armstrong公理（推理规则）：

(1) 反射律：如果 $Y \subseteq X$ ，则 $X \rightarrow Y$

(2) 增广率：如果 $X \rightarrow Y$ ，则 $XZ \rightarrow YZ$

(3) 传递律：如果 $X \rightarrow Y$ 、 $Y \rightarrow Z$ ，则 $X \rightarrow Z$

推论：

(4) 合成规则：如果 $X \rightarrow Y$ 、 $X \rightarrow Z$ ，则 $X \rightarrow YZ$

(5) 分解规则：如果 $X \rightarrow YZ$ ，则 $X \rightarrow Y$ 、 $X \rightarrow Z$

(6) 伪传递规则：如果 $X \rightarrow Y$ 、 $YW \rightarrow Z$ ，则 $XW \rightarrow Z$

求函数依赖集D的闭包：

对 $S \langle A, D \rangle$ ， $A = \{a, b, c, g, h, i\}$ ， $D = \{a \rightarrow b, a \rightarrow c, cg \rightarrow h, cg \rightarrow i, b \rightarrow h\}$ 。  
计算 $(ag)_{D^+}$

属性的闭包定理：设D 为属性集A上的一组函数依赖， $X \rightarrow Y$  能根据Armstrong公理导出的充分必要条件是Y 属于  $X D^+$ 。

| 所用依赖 | 初始                 | $(ag)_{D^+}$          |
|------|--------------------|-----------------------|
|      | $a \rightarrow b$  | ag                    |
|      | $a \rightarrow c$  | agb                   |
|      | $cg \rightarrow h$ | agbc                  |
|      | $cg \rightarrow i$ | agbch                 |
|      |                    | agbchi                |
|      |                    | $(ag)_{D^+} = agbchi$ |

### 函数依赖集等价

定义：如果  $G^+ = D^+$ ，则称  $D$  与  $G$  等价。

等价：  $D$  中的每个函数依赖于  $G^+$ ，且  $G$  中的每个函数依赖于  $D^+$ 。

### 最小依赖集

(1)  $D$  中任一函数依赖的右部仅含有一个属性。

(2)  $D$  中不存在这样的函数依赖  $X \rightarrow A$ ， $X$  有真子集  $Z$  使得  $D - \{X \rightarrow A\} \cup \{Z \rightarrow A\}$  与  $D$  等价。

(3)  $D$  中不存在这样的函数依赖  $X \rightarrow A$ ，使得  $D$  与  $D - \{X \rightarrow A\}$  等价。

极小化过程：

(1) 逐个检查函数依赖的右端，如果有多余一个的，拆分

(2) 逐个取出函数依赖，逐个检查左端的每个元素去除后是否影响函数依赖成立

(3) 逐个检查函数依赖是否多余

(最小依赖集不唯一)

### 2. 模式分解

模式分解有两方面需要关注：是否是无损连接，是否保持依赖

模式分解就是将模式中的属性分成  $n$  部分，其中任意一个部分不是另一个部分的子集。

如果无法利用新表恢复原表，那就会丢失信息。（链接后的元组比原来的多）

#### (1) 无损连接分解

对于模式  $S$ ，分解为  $n$  个子模式之后， $t$  是  $S$  上的任意关系， $t_i = \Pi_{S_i}(t)$ ，则： $t \subseteq t_1 \bowtie t_2 \bowtie \dots \bowtie t_n$

若： $t = t_1 \bowtie t_2 \bowtie \dots \bowtie t_n$  则为无损连接分解

#### 无损连接检验算法

列一个表，原模式每个属性为一列，每个子模式一行。

首先，对于每一行，对于有的属性，标为  $x$  列数，否则标为  $y$  行列

对于每个函数依赖，检查：左属性全相等的行，将右属性改为全相等（有  $x$  为  $x$ ，无  $x$  为最上面的）

如果一行全  $x$ ，那么无损

对于  $n=2$  的情况，无损连接分解等价于模式的交可以推出其中一个模式（即：超键）

定理：设  $\rho = (R_1, R_2)$  是  $R$  的一个分解， $D$  是  $R$  上的函数依赖集，分解  $\rho$  具有无损连接性的充分必要条件是： $R_1 \cap R_2 \rightarrow (R_1 - R_2) \in D^+$  或  $R_1 \cap R_2 \rightarrow (R_2 - R_1) \in D^+$

#### (2) 保持函数依赖的模式分解

首先定义函数依赖集合  $D_i = \{X \rightarrow Y \mid X \rightarrow Y \in D^+ \wedge XY \subseteq A_i\}$  称为  $D$  在  $A_i$  上的投影

如果对于一个函数分解，函数依赖集合  $D$  等价于所有投影的并，则称保持函数依赖的。

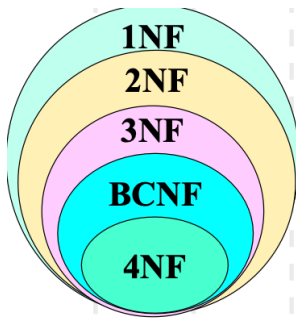
### 3. 范式

完全函数依赖：左没有任何一个真子集可以决定右，则右对左完全函数依赖  $A \xrightarrow{F} B$

部分函数依赖：反之，则右对左部分函数依赖  $A \xrightarrow{P} B$

传递函数依赖：如果  $A \rightarrow B$ ， $B \rightarrow C$ ， $B \not\rightarrow A$ ， $B \not\subseteq A$ ， $C \not\subseteq B$ ，则称  $A \xrightarrow{T} C$  是传递依赖

直接函数依赖：反之，则右对左直接函数依赖  $A \xrightarrow{D} B$



第一范式：每个关系 $r$ 的属性值都是不可分的原子值

第二范式：若 $S \in 1NF$ ，且每一个非主属性完全函数依赖于候选键，则 $R \in 2NF$ 。

（采用投影分解法将一个1NF的关系分解为多个2NF的关系，从而消除原1NF关系中因非主属性对键部分依赖存在的问题）

第三范式：关系模式 $S \langle A, D \rangle$ 满足第二范式，且它的任何一个非主属性都不传递依赖于任何候选键

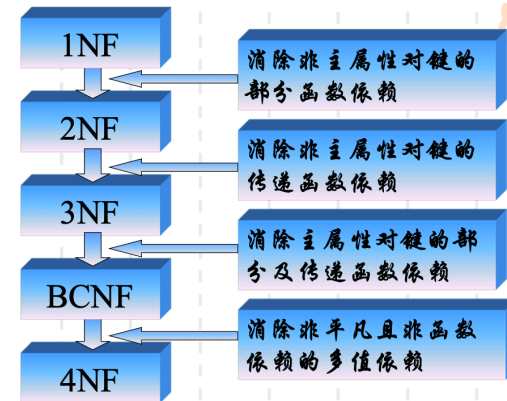
（第三范式中，每一个非主属性既不部分依赖于键也不传递依赖于键。）

（采用投影分解法将一个2NF的关系分解为多个3NF的关系，消除因传递依赖而存在的问题）

BC范式：满足第三范式，且它的任何一个主属性既不部分也不传递依赖于任何候选键

（采用投影分解法将一个3NF的关系分解为多个BCNF的关系，可以解决原3NF关系中因主属性对键的部分或传递依赖而存在的问题）

一个低一级范式的关系模式，依其中的数据依赖、通过模式分解可以转换为若干个高一级范式的关系模式的集合，这种过程就叫规范化。



#### 4. 规范化

重要事实：

若要求分解具有无损连接性，那么模式分解一定能够达到BCNF（4NF，5NF）

若要求分解既具有无损连接性，又保持函数依赖，则模式分解一定能够达到3NF，但不一定能够达到BCNF

无损分解且保持依赖地分解成3NF模式集：

首先，判断是否为3NF，若不是：

生成最小依赖集，对每个函数依赖，两边属性并集生成模式

然后查看候选键是否被包含（在同一个模式中），如果不被包含，键单独作为一个模式

达到BCNF无损连接分解算法：

首先，判断是否为BCNF，若不是：

取非平凡函数依赖的左部不含键的，合并左右两边得到新模式

让原模式减去右部属性，得到新的模式，返回判断BCNF循环

## 9. 查询处理与优化

### 1. 查询优化

基本原则：中间结果最小

- 尽可能早地执行选择运算
- 尽可能早地执行投影运算
- 将选择和笛卡儿积运算组成自然联接
- 较小的关系作为联接运算的左参数

转换规则：

自然连接、并、交、笛卡儿积的交换律和结合律

## 选择上的转换规则

$$\checkmark \sigma_{C_1 \wedge C_2}(E) = \sigma_{C_1}(\sigma_{C_2}(E))$$

$$\checkmark \sigma_{C_1}(\sigma_{C_2}(E)) = \sigma_{C_2}(\sigma_{C_1}(E))$$

选择尽可能早做

$$\sigma_p(E_1 \bowtie E_2) = [\sigma_p(E_1)] \bowtie E_2$$

可推出另一些规则

$$\sigma_q(E_1 \bowtie E_2) = E_1 \bowtie [\sigma_q(E_2)]$$

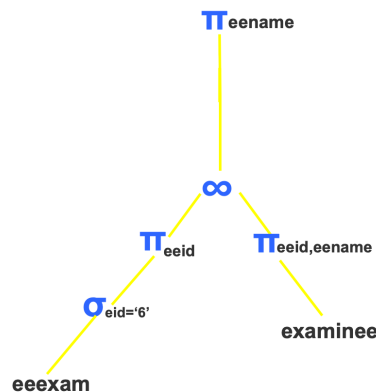
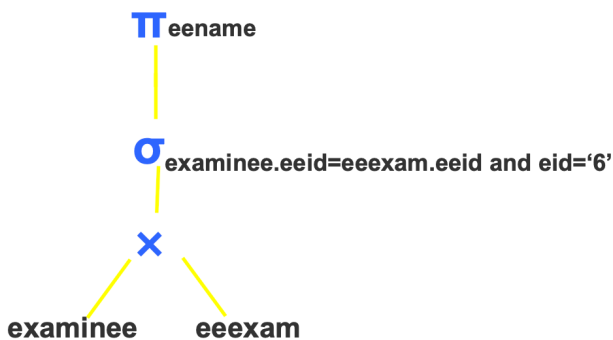
## 投影上的缩减规则

$$\checkmark \Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

$$\sigma_{p \wedge q}(E_1 \bowtie E_2) = [\sigma_p(E_1)] \bowtie [\sigma_q(E_2)]$$

$$\sigma_{p \wedge q \wedge m}(E_1 \bowtie E_2) = \sigma_m[\sigma_p(E_1) \bowtie \sigma_q(E_2)]$$

$$\Pi_x[\sigma_p(S)] = \Pi_x \{ \sigma_p[\Pi_{xz}(S)] \}$$



选择操作：

对于小关系，使用全表顺序搜索，即使有索引列

对于大关系，估算查询结果的元组数目，如果比例较小使用索引扫描，否则全表顺序扫

表达式的求值方法与优化

查询优化的方法：中间结果传递

(1) 实体化计算方法：每次计算的结果被保存（实体化）到一个临时关系中

(2) 流水线计算方法：直接作为输入传送到下一个运算

优化：哪些运算可以用流水线

数据库特征参数：

估计运算花费

假设R为一关系

- ✓ 关系的基：指关系R包含的元组个数，记为Card(R)
- ✓ 属性的长度：指属性A定义的取值字节数，记为Length(A)
- ✓ 元组的长度：关系R中每个元组的字节数，记为Length(R)，  
Length(R) =  $\sum \text{Length}(A_i)$
- ✓ 关系的大小：关系R所包含的字节数，记为Size(R)  
Size(R) = Card(R) \* Length(R)
- ✓ 属性的特征值：指关系R中属性A取值不同的属性值个数，记为Val(A)
- ✓ 属性A的值域：记为Dom(A)
- ✓ 属性A的最大值和最小值：记为Max(A)和Min(A)

## 10.事务处理

### 1.事务概念与性质

事务是由一系列数据库操作序列构成的程序执行单元。

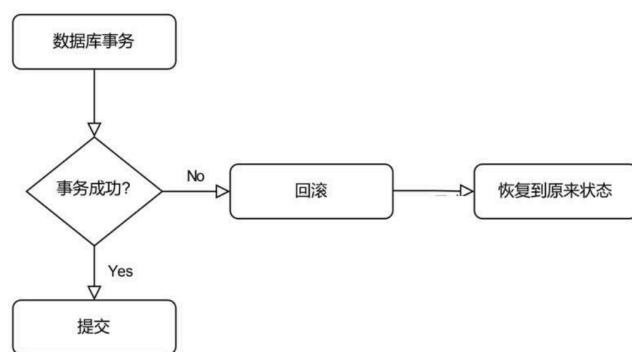
事务的性质：原子性、一致性、隔离性、持久性

原子性强调了一个事务是一个逻辑工作单元，是一个整体，是不可分割的。一个事务所包含的操作要么全部做，要么全部不做。

一致性：一个事务执行对数据库操作，事务将使数据库从一种一致性的状态变换成另一种一致性状态。

隔离性：如果每个事务单独执行能保持原子性和一致性，这些事务并发执行也能保持原子性和一致性，每个事务感觉不到其它事务的存在，则是事务的隔离性。（并发控制协议）

事务的持久性是指一旦事务成功完成，该事务对数据库所施加的所有更新都是永久的。



## 2.并发执行与调度

事务的执行顺序称为一个调度，表示事务的指令在系统中执行的时间顺序。

在串行调度中，属于同一事务的指令紧挨在一起；在并发调度中，来自不同事务的指令可以交叉执行。一个事务中指令的顺序必须保持不变。

当并发调度等价于某个串行调度时，它是正确的

并发执行可能会出现的问题：

丢失修改、无法重复读、读脏数据、

冲突：

当两条指令是不同事务在相同数据项上的操作，并且其中至少有一个是write指令时，则称这两条指令是冲突的。（非冲突指令交换次序不会影响调度的最终结果。）

冲突等价：如果调度H可以经过一系列非冲突指令交换转换成调度H'，则称调度H与H'是冲突等价的

冲突可串行化：当一个调度H与一个串行调度冲突等价时，则称该调度是冲突可串行化的

优先图：顶点集由所有参与调度的事务组成，边集由满足下述条件之一的边 $\tau_i \rightarrow \tau_j$ 组成：

- ①在 $\tau_j$ 执行read(z)之前， $\tau_i$ 执行write(z)
- ②在 $\tau_j$ 执行write(z)之前， $\tau_i$ 执行read(z)
- ③在 $\tau_j$ 执行write(z)之前， $\tau_i$ 执行write(z)

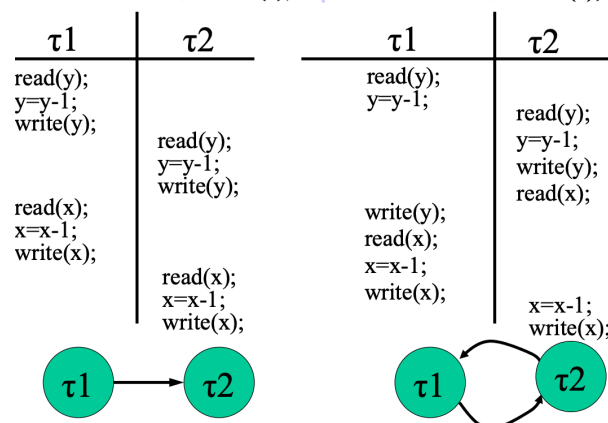
冲突可串行化判定：

如果优先图中存在边 $\tau_i \rightarrow \tau_j$ ，则在任何等价于H的串行调度H'中， $\tau_i$ 都必须出现在 $\tau_j$ 之前。如果调度H的优先图中有环，则调度H是非冲突可串行化的。如果图中无环，则调度H是冲突可串行化的。

可以得出串行拓扑序

## 冲突可串行化调度

| $\tau_1$              | $\tau_2$              | $\tau_1$              | $\tau_2$                                       |
|-----------------------|-----------------------|-----------------------|------------------------------------------------|
| read(y);<br>write(y); |                       | read(y);<br>write(y); |                                                |
|                       | read(y);<br>write(y); |                       | read(y);                                       |
| read(x);<br>write(x); |                       | read(x);              | write(y);                                      |
|                       | read(x);<br>write(x); | write(x);             | read(x);<br>write(x);                          |
| read(y);<br>write(y); |                       | read(y);<br>write(y); |                                                |
|                       | read(y);<br>write(y); | read(x);<br>write(x); |                                                |
| read(x);              |                       |                       | read(y);<br>write(y);                          |
|                       | read(x);<br>write(x); |                       | read(x);<br>write(x);                          |
| write(x);             |                       |                       | read(y);<br>write(y);<br>read(x);<br>write(x); |



3.并发控制

(1) 封锁：一个事务对某个数据对象加锁，取得对它一定的控制，限制其它事务对该数据对象使用要访问一个数据项z，事务ti必须先申请对z的封锁，如果z已经被事务tj加了不相容的锁，则ti需要等待，直至tj释放它的封锁

排它锁（X锁）

事务τ对数据对象z加上X锁，则其它事务对z的任何封锁请求都不能成功，直至τ释放z上的X锁；又称写锁（事务中涉及z的写操作时应申请写锁）

申请对z的排它锁：Xlock(z)

共享锁（S锁）

事务τ对数据对象z加上S锁，则其它事务对z的X锁请求不能成功，而对z的S锁请求可以成功；又称读锁(事务中只涉及对z的读操作时应申请读锁)

申请对z的共享锁：Slock(z)

| 相容矩阵comp(A,B) |        |   |
|---------------|--------|---|
| 请求锁模式A        | 现有锁模式B |   |
|               | S      | X |
| S             | 是      | 否 |
| X             | 否      | 否 |

两阶段封锁协议(简称2PL)是最常用的一种封锁协议，理论上证明使用两阶段封锁协议产生的是可串行化调度。

增长阶段：事务可以获得锁，但不能释放锁

缩减阶段：事务可以释放锁，但不能获得锁

封锁粒度：封锁对象单元的大小

小粒度锁：元组锁（行级锁）

中粒度锁：页级锁（块级锁）

大粒度锁：关系锁（表级锁）

细粒度锁比粗粒度锁使事务具有更好的并发度

事务的隔离性：

冲突可串行化能实现事务隔离性，保障事务并发处理过程的数据一致性。

冲突可串行化是充分条件而非必要条件，严格的可串行化调度会影响事务的并发，甚至会退化成单纯的串行化调度，从而影响性能。

长期锁：锁被保持到事务提交的时候 短期锁：锁只保持到语句执行完毕

所有的隔离级别上写锁都是长期锁，每个级别上读锁的处理方式不同。

4种隔离级别：

| 隔离级别 | 脏读  | 不可重复读 | 幻读  |
|------|-----|-------|-----|
| 读未提交 | 允许  | 允许    | 允许  |
| 读已提交 | 不允许 | 允许    | 允许  |
| 可重复读 | 不允许 | 不允许   | 允许  |
| 可串行化 | 不允许 | 不允许   | 不允许 |

### (1) 读未提交

事务没有获得读锁也可以执行读操作。

事务可以读取被其他事务已经加上写锁的数据。

事务可能会读取没有提交事务所写的脏数据。

### (2) 读提交

事务读数据前要获得数据对象上的短期读锁。

事务只会读取其他事务提交后的数据，不会读取脏数据。

事务对同一数据对象进行的再次读取操作，出现不可重复读问题。

### (3) 可重复读

事务要获取SELECT语句查询结果中每个元组上的长期读锁。

事务对查询结果中元组的再次查询不存在不可重复读问题。

事务中该SELECT语句的再次执行有可能检索到前次查询没有检索到的新元组。

### (4) 可串行化

事务读操作前要获得数据对象上的长期读锁。

事务的查询操作不会有幻像现象。

事务的执行是可串行化的。

死锁：两个事务都封锁了一些数据对象，并相互等待对方释放另一些数据对象以便对其封锁，结果两个事务都不能结束，则发生死锁

死锁处理方法：

#### (1) 时间戳：由数据库创建的用于标识事务串行化顺序的标识符

事务只能访问该事务时间戳前面的数据项而不能访问后面的数据项，从而所有事务有单调顺序

每个读写操作按照所在事务的时间戳排序执行

T只能读T之前事务写的的数据，不能读T之后写的的数据

T只能写T之前事务读写的的数据，不能写T之后读写的的数据

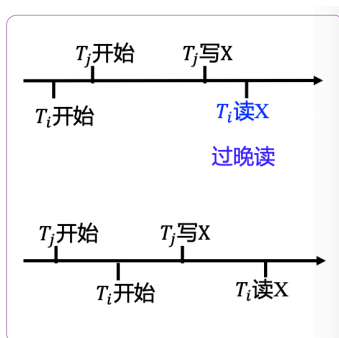
数据项的时间戳

**W\_TS X**：对X成功执行写操作的所有事务的最大时间戳

**R\_TS X**：对X成功执行读操作的所有事务的最大时间戳

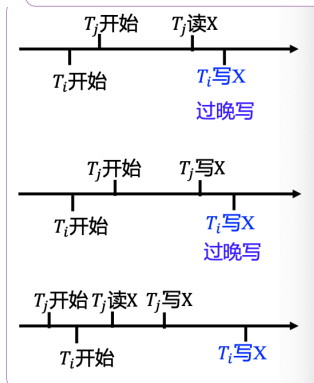
#### 读操作：事务 $T_i$ 读数据项X

- 若 $TS(T_i) < W\_TS(X)$ ，则事务 $T_i$ 需要读取的X值已被其他事务新值覆盖，拒绝读操作，事务 $T_i$ 被回滚（回滚该事务涉及的所有读写操作）
- 若 $TS(T_i) \geq W\_TS(X)$ ，则可执行读操作，事务 $T_i$ 在本地保存当前的X值以便于可重复读。更新 $R\_TS(X)$ 的值为 $R\_TS(X)$ 和 $TS(T_i)$ 中的较大值



#### 写操作：事务 $T_i$ 写数据项X

- 若 $TS(T_i) < R\_TS(X)$ ，则X的当前值已经被其他更晚的事务（时间戳更大的）读取过，所以拒绝此次写操作，并回滚事务 $T_i$
- 若 $TS(T_i) \geq R\_TS(X)$ 且 $TS(T_i) < W\_TS(X)$ ，则X的当前值是其他更晚事务修改后的结果。 $T_i$ 此时的写操作是在用旧值覆盖新值，所以拒绝此次写操作，并回滚事务 $T_i$
- 若 $TS(T_i) \geq R\_TS(X)$ 且 $TS(T_i) \geq W\_TS(X)$ ，事务 $T_i$ 可以执行写操作，并在本地保存当前的X值以便于可重复读。将 $W\_TS(X)$ 更新为 $TS(T_i)$ 。



饿死：一系列短事务与某个开始较早但执行时间较长的事务冲突时，长事务不断的重启而饿死。  
 解决办法：如果某个事务反复重启，则暂停与之冲突的其他事务，使该事务正常完成。

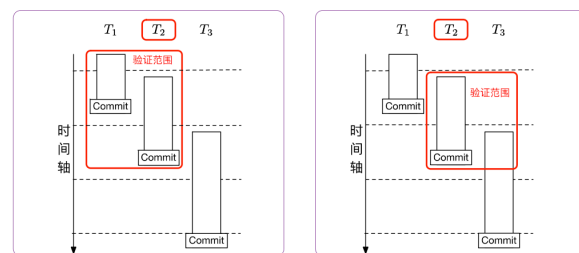
乐观并发控制协议：

（事务冲突较少、执行时间较短）

验证阶段有效性验证范围：

向后验证：验证当前事务与已提交的事务之间是否存在冲突

向前验证：验证当前事务与未提交事物是否存在冲突



#### 4.故障恢复

##### (1) 故障分类

事务故障：运算溢出

系统故障：内存信息丢失，但未破坏外存中数据如CPU故障、突然停电

磁盘故障：如磁盘的磁头碰撞、瞬时的强磁场干扰

##### (2) 共性基础措施

备份+日志

备份：将数据库复制到磁带或另一个磁盘上保存起来

日志：记录数据库的每一次更新活动

日志内容包括：记录名、旧记录值、新记录值、事务标识符、操作标识符等

事务 $\tau_i$ 开始时，写日志项： $\langle \tau_i \text{ start} \rangle$

事务 $\tau_i$ 执行更新前，写日志项： $\langle \tau_i, \text{erid}, \text{Vold}, \text{Vnew}, \text{U} \rangle$ ，Vold 是X更新前的值，Vnew 是X更新后的值，U表示操作是更新

事务 $\tau_i$ 结束后，写日志项： $\langle \tau_i \text{ commit} \rangle$

原则：对于尚未提交的事务，在将DB缓冲区写到外存之前，必须先将日志缓冲区内容写到外存去

圆满事务：日志文件中记录了事务的commit标识

夭折事务：日志文件中只有事务的Begin transaction标识，无commit

对圆满事务所做过的修改操作应执行redo操作，即重新执行该操作，修改对象被赋予新记录值

对夭折事务所做过的修改操作应执行undo操作，即撤消该操作，修改对象被赋予旧记录值

$\langle \tau_0 \text{ start} \rangle$   
 $\langle \tau_0, x, 1000, 950, U \rangle$   
 $\langle \tau_0, y, 2000, 2050, U \rangle$   
 $\langle \tau_0 \text{ commit} \rangle$   
 $\langle \tau_1 \text{ start} \rangle$   
 $\langle \tau_1, z, 700, 600, U \rangle$   
 $\tau_0$ 圆满事务， $\tau_1$ 夭折事务

##### (3) 事务故障修复—撤消事务已对数据库所做的修改

反向扫描日志文件，查找该事务的更新操作，执行逆操作，即将事务更新前的旧值写入数据库继续

反向扫描日志文件，查找该事务的其他更新操作，并做同样处理

#### 系统故障恢复

正向扫描日志文件，找出圆满事务，记入重做队列；找出夭折事务，记入撤消队列

反向扫描日志，对撤消队列中事务 $\tau_i$ 的每一个日志记录执行undo操作

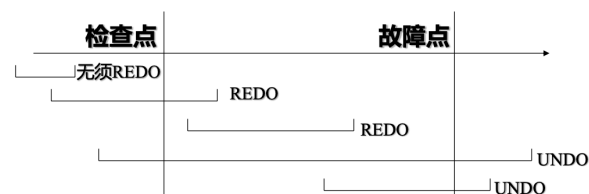
正向扫描日志文件，对重做队列中事务 $\tau_i$ 的每一个日志记录执行redo操作

检查点：

原理：保证在检查点时刻日志与数据库内容是一致的

步骤：

在日志文件中找到最后一个检查点记录



由该检查点记录得到检查点建立时刻所有正在执行的事务清单ACTIVE-LIST，建立两个事务队列UNDO-LIST、REDO-LIST

把ACTIVE-LIST暂时放入UNDO-LIST队列，REDO队列暂为空。

从检查点开始正向扫描日志文件，直到日志文件结束：

如有新开始的事务 $\tau_i$ ，把 $\tau_i$ 暂时放入UNDO-LIST队列

如有提交的事务 $\tau_j$ ，把 $\tau_j$ 从UNDO-LIST队列移到REDO-LIST队列

对UNDO-LIST中的每个事务执行UNDO操作；对REDO-LIST中的每个事务执行REDO操作

## 磁盘故障恢复

装入最新的数据库后备副本，使数据库恢复到最近一次备份时的一致性状态

装入相应的日志文件副本，重做已完成的事务

## 5.PG数据保护

### 1.数据安全

数据保密性：数据内容保密、数据存在保密

数据完整性：指数据的可信度，通常保护完整性就是指仅允许合法授权的数据修改

数据可用性：对数据的期望使用能力

描述系统访问策略的简单模型：访问控制矩阵。行为角色，列为对象，元素表示相应用户对相应数据库对象的访问权限。

完整性约束：满足明确定义的约束的数据才可以进入数据库

### 2.视图

视图使不同用户从多种角度看待统一数据

```
CREATE VIEW <视图名> [(<列名> [, <列名>]...)] AS
    <子查询>
WITH CHECK OPTION;
```

| eeexamv1                  |        |       |         |
|---------------------------|--------|-------|---------|
| eeid                      | eedepa | ename | achieve |
| CREATE VIEW eeexamv2 AS   |        |       |         |
| SELECT eeid,ename,achieve |        |       |         |
| FROM eeexamv1             |        |       |         |
| WHERE eedepa='历史学院';      |        |       |         |

也可以使用视图定义视图

视图和表都是关系，区别：存储表的定义和数据，存储视图的定义、不存储数据，临时计算数据

PG只允许对可更新视图进行修改，可更新视图需要满足一些条件

### 3.访问控制

访问控制：给定用户拥有在给定数据库对象上的给定操作权限。

postgres

创建具有超级权限的角色用语句

CREATE ROLE yuxiaotong LOGIN

创建可创建角色的角色用语句

CREATE ROLE lichen CREATEROLE

创建数据库超级用户角色用语句

CREATE ROLE nini SUPERUSER

创建角色用语句

CREATE ROLE yanni

创建可创建数据库的角色用语句

CREATE ROLE masu CREATEDB

创建具有口令的角色用语句

CREATE ROLE wangxi PASSWORD '654321'

超级用户有权创建超级用户



一旦主角色存在了，就可以用 GRANT和REVOKE命令添加和撤销权限，不可以建立循环的成员关系。

#### 4.完整性约束

主键约束：各元组主键值不能重复，不能为空

外键约束：外键的值必须是其值之一

值非空：not null

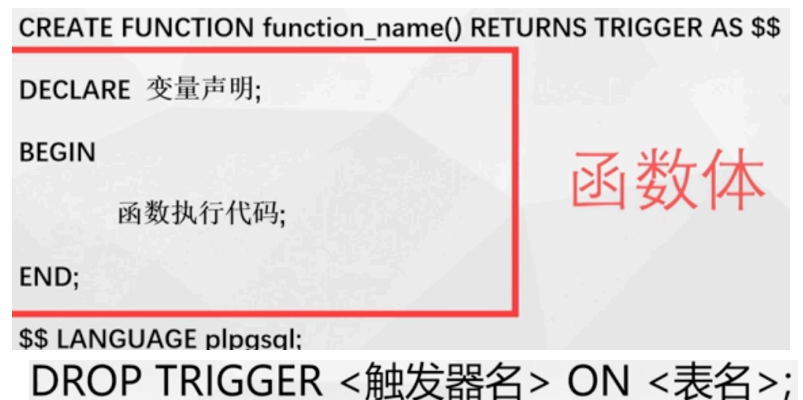
值唯一：unique



#### 5.触发器

用户定义在关系表上的由事件驱动调用函数的机制。

语句级触发器执行一次，行级触发器有个数次



删除触发器：

#### 6.事务

事务是对数据库进行操作的程序单位，具有原子性、一致性、隔离性、持久性

#### 7.加密

#### 10.大数据技术

##### 1.大数据及其特征

大数据的特征：数据量大、数据类型多样、数据产生和需要处理的速度快、价值高而密度稀疏  
大数据技术需求：可伸缩性、可靠性、高效性

## 2.大数据存储技术

由于大数据的数据规模远远超过单节点存储能力，通常在计算机集群中使用分布式文件系统来存储  
主从结构

## 4.大数据应用—推荐

推荐方法：基于内容推荐/协同过滤推荐（基于用户的协同过滤/基于物项的协同过滤）

基于用户的协同过滤：根据已知信息得到用户间的相似性，找到用户邻居，把邻居感兴趣的物项推荐给用户。